DART FOR FLUTTER PROGRAMMING FOR MOBILE APPS

LAB 4

Consuming APIs in Flutter

Student Information

Registration No:	22302746
Campus:	UR-CST-GAKO
Deadline:	Gako Campus — Sunday Midnight
Academic Year:	2025 – 2026

Reg No: 223027476 UR-CST-6480

LAB 4: Consuming APIs in flutter

1. Dependencies used and why?

- http^1.1.0
  - This is the main dependency for making HTTP requests
  - It provides GET(), POST(), PUT() and DELETE() methods
  - Essential for communicating with JSONplaceholder API
  - Returns Future objects that work with flutter's async/await pattern
  - I chose it because it's the official flutter HTTP package, well documented, and ~~lightweight~~ lightweigh.

2. fluttertoast: ^8.2.5
  - Shows beautiful toast notifications when posts are created, updated, or deleted.
  - provides colored message (green for success, blue for update, orange for delete)
  - Give users clear feedback on their actions.

3. Shimmer: ^3.0.0
  - Creates loading shimmer effect like facebook/Instagram.
  - Improves user experience while waiting for post to load
  - Shows that content is loading rather than leaving a blank screen.

Why I chose these:

- http is the official flutter package, well-documented and maintained.
- fluttertoast and Shimmer enhance UI without adding complexity
- ALL are lightweigh and don't affect app performance

CamScanner

2. How I handled API exceptions (network errors, invalid data)?

I handled API exceptions using try-catch blocks with specific exception types:

1. SocketException - occurs when there is no internet connection.
   - Show message: "No internet connection. please check your network."
   - User sees a Retry button to try again.

2. TimeoutException
   - Occurs when server takes too large to respond (more than 10 seconds)
   - Show message: "Connection timeout. Please try again."
   - User can tap Retry to attempt the request again

3. FormatException
   - Occurs when response data is not valid JSON format
   - Show message: "Invalid data format received from server."
   - Log the error for debugging

4. HTTP Status Codes
   - 200/201 = Success → process the data.
   - 404 = Not found → show "post not found"
   - 403 = Forbidden → show "access denied" (normal for test APIs)
   - 500 = Server error → shows "server error, please try again later"

5. General Exception
   - Catch any other unexpected errors
   - Show generic message: "Unexpected error occured"

UI Error handling:

I use FutureBuilder which automatically shows:
- Loading spinner while waiting
- Error message with retry button when something fails.
- The data when the request succeeds.

CamScanner

3. Future widget and the part of the UI depends on the future

⊕ What is Future? A Future represents a value that will be available later. API calls take time, so they return Future to keep the app responsive.

⊕ What is FutureBuilder? A Flutter widget that listens to a future and rebuilds the UI when it completes. It provides a snapshot with connectionState, hasData, hasError, data, and data error.

Three UI states

① Loading - When waiting for data -> Show Shimmer effect

② Error - When snapshot.hasError is true -> Show error message + Retry button.

③ Success - When snapshot.hasData is true -> Show list of posts.

This pattern ensures users always see appropriate feedback: Loading indicator while waiting, error message with retry option when something fails and actual content when it arrives.

CamScanner

Flutter Lab 4

Posts Manager — App Screenshots

Gako Campus | 2026

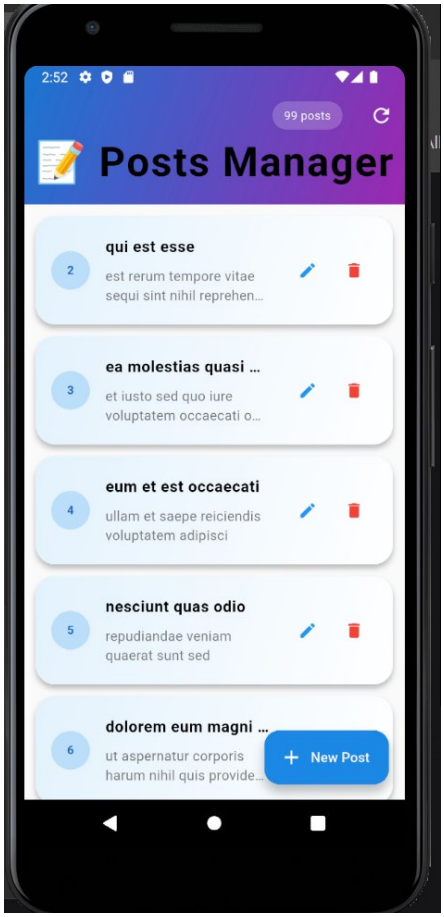
Screenshots Summary

#	Screenshot	Feature Tested
1	Home Screen — All Posts	READ ALL — GET /posts
2	Loading State	FutureBuilder — ConnectionState.waiting
3	Detail Screen — Read Post	READ ONE — GET /posts/{id}
4	Create Form — Empty	CREATE — POST /posts
5	Post Created Successfully	CREATE — 201 Created Response
6	Edit Form — Pre-filled	UPDATE — PUT /posts/{id}
7	Post Updated Successfully	UPDATE — 200 OK Response
8	Delete Confirmation Dialog	DELETE — Confirmation Before Action
9	Post Deleted Successfully	DELETE — DELETE /posts/{id}
10	Error State — No Internet	ERROR HANDLING — ClientException

Screenshot 1

Home Screen — All Posts

The main screen displaying all posts fetched from the JSONPlaceholder API. Each post shows its ID, title, body preview, and edit/delete action buttons.

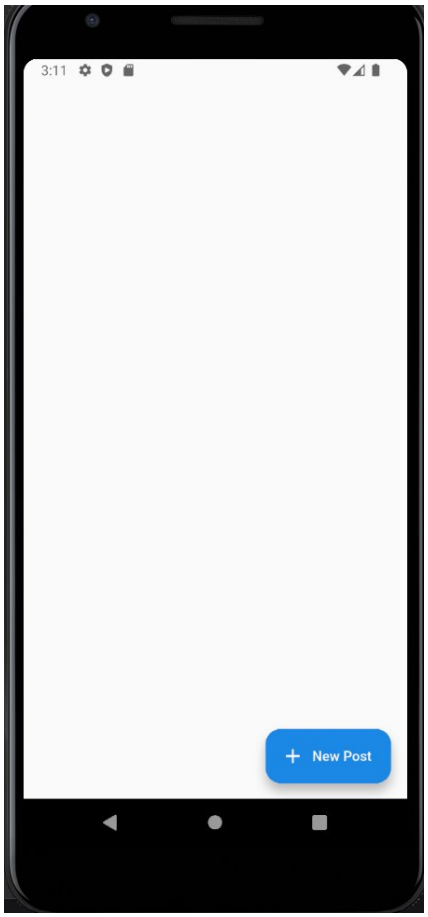


Feature: **READ ALL — GET /posts**

Screenshot 2

Loading State

FutureBuilder showing the loading state (CircularProgressIndicator) while the app waits for the API response. This demonstrates async data fetching.



Feature: **FutureBuilder — ConnectionState.waiting**

Screenshot 3

Detail Screen — Read Post

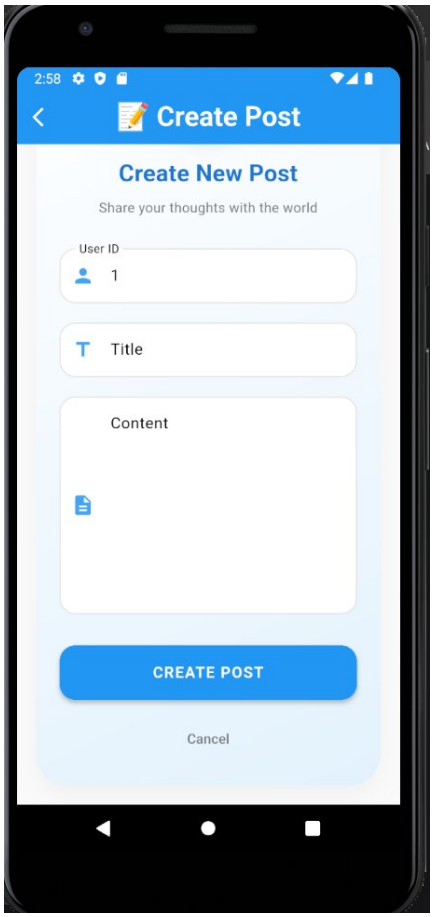
Detailed view of a single post showing the full title, body content, User ID, and Post ID. Accessed by tapping any post in the list.

Feature: **READ ONE — GET /posts/{id}**

Screenshot 4

Create Form — Empty

The form for creating a new post with empty fields for User ID, Title, and Content. Input validation ensures no field is left blank.

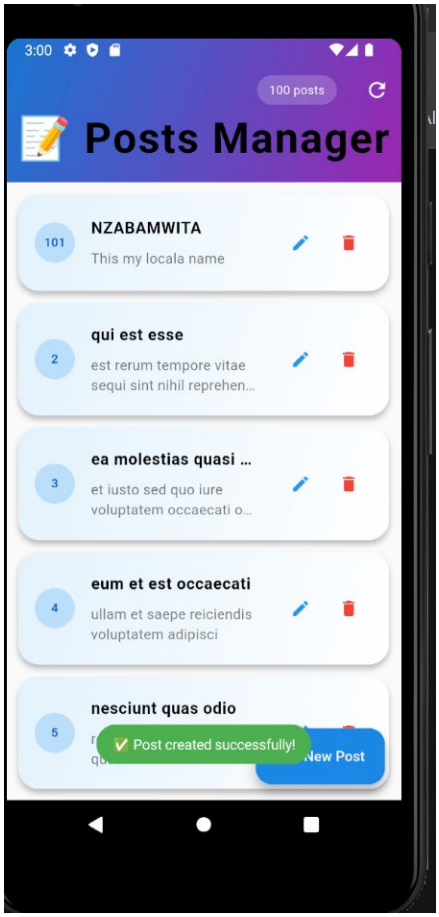


Feature: **CREATE — POST /posts**

Screenshot 5

Post Created Successfully

After submitting the create form, a success toast message appears and the new post (NZABAMWITA) appears at the top of the list.

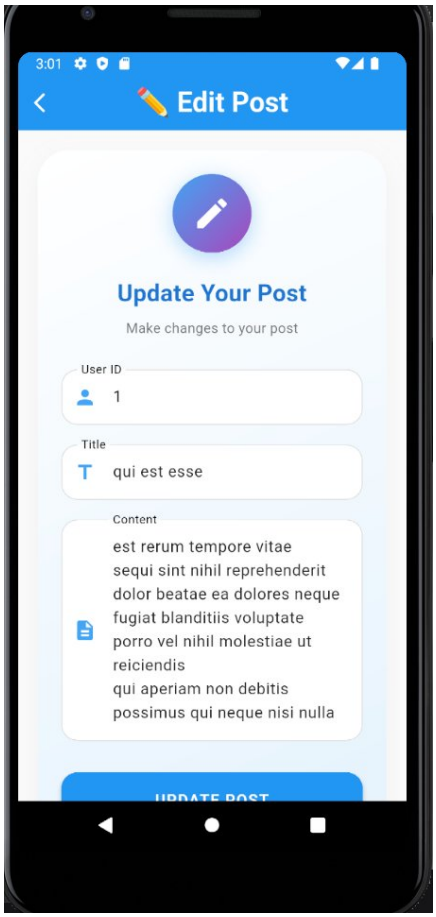


Feature: **CREATE — 201 Created Response**

Screenshot 6

Edit Form — Pre-filled

The edit form pre-populated with the existing post data. The user can modify the title and content, then submit the update.

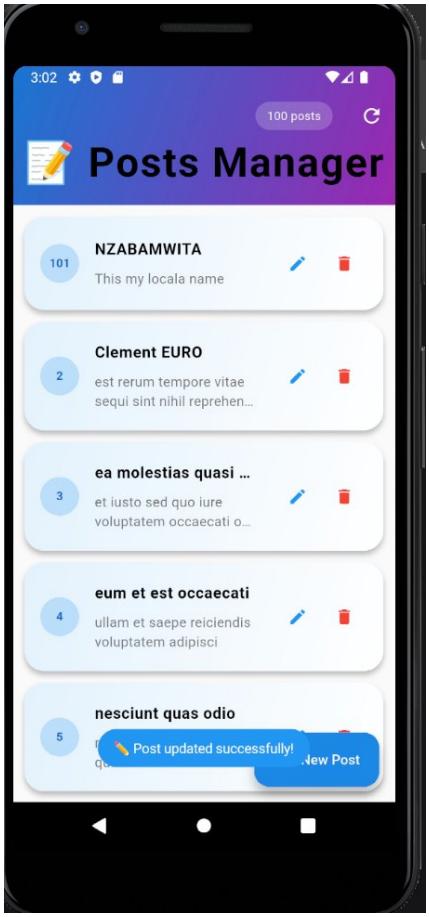


Feature: **UPDATE — PUT /posts/{id}**

Screenshot 7

Post Updated Successfully

After submitting the edit form, a success toast message appears confirming the post was updated. The updated title (Clement EURO) is visible in the list.

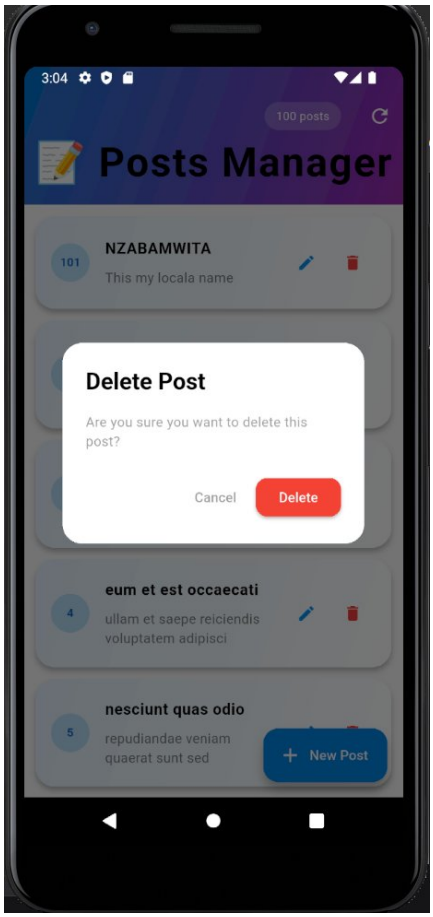


Feature: **UPDATE — 200 OK Response**

Screenshot 8

Delete Confirmation Dialog

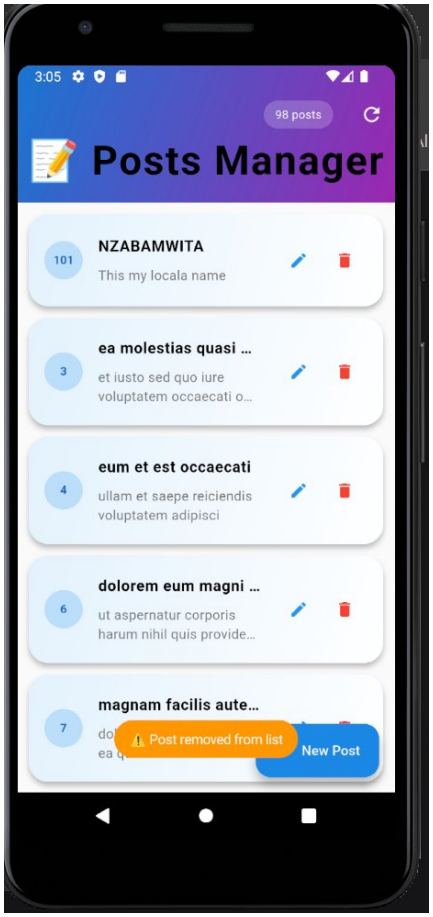
A confirmation dialog appears when the user taps the delete button, asking "Are you sure you want to delete this post?" with Cancel and Delete options.



Feature: **DELETE — Confirmation Before Action**

Screenshot 9 Post Deleted Successfully

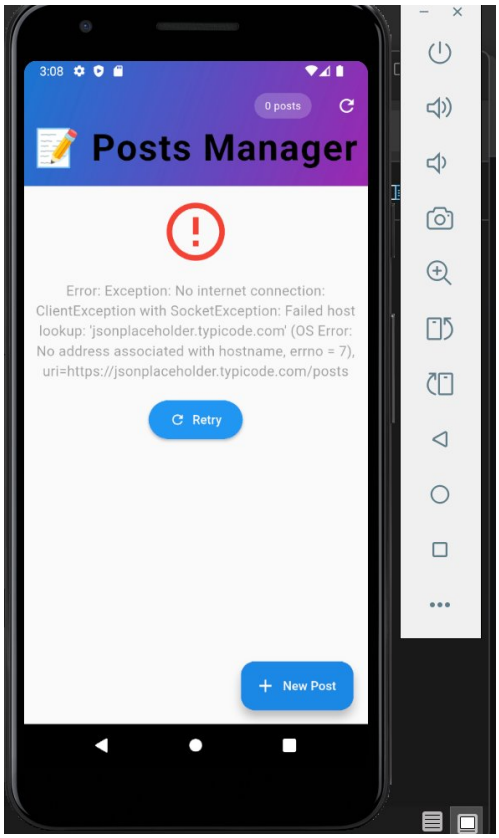
After confirming deletion, a toast message shows "Post removed from list" and the post count decreases from 100 to 98, confirming the deletion.



Feature: **DELETE — DELETE /posts/{id}**

Screenshot 10 Error State — No Internet

When internet is unavailable, the app shows a clear error message with a Retry button. This demonstrates ClientException handling for network failures.



Feature: **ERROR HANDLING — ClientException**

All 10 Screenshots Captured Successfully

Flutter Lab 4 | Posts Manager | Gako Campus | 2026

CRUD Operations Demonstrated

Operation	HTTP Method	Screenshot	Status
Read All Posts	GET /posts	Screenshot 1, 2	✓ Done
Read One Post	GET /posts/{id}	Screenshot 3	✓ Done
Create Post	POST /posts	Screenshot 4, 5	✓ Done
Update Post	PUT /posts/{id}	Screenshot 6, 7	✓ Done
Delete Post	DELETE /posts/{id}	Screenshot 8, 9	✓ Done
Error Handling	N/A	Screenshot 10	✓ Done